

Informe Final de Proyecto

Tema: Informe Consolidado Final - Sistema Café-Sys

Nota

Estudiantes	Escuela	Asignatura
Gustavo Alonso Carrillo Villalta Fredy José Aragón Carpio Diego Marcelo Arce Coaquira José Manuel Bravo Rojas	Escuela Profesional de Ingeniería de Sistemas	Desarrollo de Aplicaciones Web Semestre: I Código: 2025004 Grupo: Desarrolladores

Entregable	Tema	Duración
Proyecto Final - Consolidado	Informe Consolidado Final - Sistema Café-Sys	Finalización de Semestre

Semestre académico	Fecha de inicio	Fecha de entrega
2026 - A	06 de Julio de 2026	13 de Julio de 2026

Dedicatoria

“Este proyecto es el resultado del esfuerzo conjunto de un equipo comprometido con la excelencia académica y el desarrollo de soluciones tecnológicas que impacten positivamente en la comunidad universitaria.”

Agradecimientos

Agradecemos a la Universidad Nacional de San Agustín de Arequipa por brindarnos las herramientas y el conocimiento necesario para llevar a cabo este proyecto. Asimismo, extendemos nuestro reconocimiento al cuerpo docente de la Escuela Profesional de Ingeniería de Sistemas por su invaluable orientación durante el desarrollo del curso de Desarrollo de Aplicaciones Web.

Resumen

El presente informe documenta el desarrollo completo de *Cafe-Sys*, una aplicación web diseñada para la gestión integral de una cafetería universitaria. El sistema aborda la problemática de la falta de automatización en los procesos de pedidos, inventario y logística de entrega, proponiendo una solución basada en una arquitectura moderna de tres capas.

La aplicación implementa un frontend desarrollado en Angular v21, un backend construido con Django y Django REST Framework, y una base de datos PostgreSQL alojada en Supabase. La comunicación entre capas se realiza mediante una API RESTful con autenticación JWT, garantizando seguridad y escalabilidad. El sistema incluye módulos de autenticación, gestión de productos y categorías, procesamiento de pedidos, administración de conductores y vehículos, logística de entregas, y paneles de reportes estadísticos para la toma de decisiones.

El proyecto se despliega utilizando contenedores Docker, con el backend en Render y el frontend en Vercel, asegurando alta disponibilidad y portabilidad del entorno. Los resultados obtenidos demuestran una reducción significativa en los tiempos de procesamiento de pedidos y una mejora en la trazabilidad del inventario.

Palabras clave: Aplicación web, Angular, Django REST Framework, PostgreSQL, API REST, JWT, Docker, Cafetería, Gestión de pedidos.

Abstract

This report documents the complete development of *Cafe-Sys*, a web application designed for the comprehensive management of a university cafeteria. The system addresses the problem of lack of automation in ordering, inventory, and delivery logistics processes, proposing a solution based on a modern three-layer architecture.

The application implements a frontend developed in Angular v21, a backend built with Django and Django REST Framework, and a PostgreSQL database hosted on Supabase. Communication between layers is carried out through a RESTful API with JWT authentication, ensuring security and scalability. The system includes authentication modules, product and category management, order processing, driver and vehicle administration, delivery logistics, and statistical reporting panels for decision-making.

The project is deployed using Docker containers, with the backend on Render and the frontend on Vercel, ensuring high availability and environment portability. The results obtained demonstrate a significant reduction in order processing times and an improvement in inventory traceability.

Keywords: Web application, Angular, Django REST Framework, PostgreSQL, REST API, JWT, Docker, Cafeteria, Order management.

Índice

1. Capítulo I: Introducción	10
1.1. Descripción del Problema	10
1.2. Objetivos	10
1.2.1. Objetivo General	10
1.2.2. Objetivos Específicos	10
1.3. Alcance	10
1.3.1. Qué hace el sistema	10
1.3.2. Qué no hace el sistema	11
1.4. Tecnologías Utilizadas	11
2. Capítulo II: Arquitectura del Sistema	12
2.1. Arquitectura General	12
2.2. Arquitectura Backend	12
2.3. Arquitectura Frontend	13
2.4. Arquitectura de Base de Datos	14
3. Capítulo III: Base de Datos	15
3.1. Diseño Conceptual (DER Conceptual)	15
3.2. Diseño Lógico y Físico (DER Detallado)	17
3.3. Descripción de Modelos	17
3.3.1. Modelo Role (Rol)	17
3.3.2. Modelo User (Usuario)	18
3.3.3. Modelo Category (Categoría)	18
3.3.4. Modelo Product (Producto)	18
3.3.5. Modelo Order (Pedido)	19
3.3.6. Modelo OrderDetail (Detalle de Pedido)	19
3.3.7. Modelo Promotions (Promociones)	20
4. Capítulo IV: Backend	21
4.1. Organización del Proyecto	21
4.2. Serializers	21
4.2.1. Serializador de Productos	21
4.2.2. Serializadores Anidados para Pedidos	21
4.3. Views y Controladores	23
4.3.1. Permisos Diferenciados y Acciones Personalizadas	23
4.4. Endpoints de la API	23
4.5. Autenticación y Seguridad	24
4.5.1. JSON Web Tokens (JWT)	24
4.5.2. Roles y Permisos	25
4.5.3. Configuración CORS	25
5. Capítulo V: Frontend	27
5.1. Arquitectura de Angular	27
5.1.1. Estructura de Componentes	27
5.2. Servicios	27
5.2.1. ApiService	27
5.2.2. AuthService	29
5.3. Rutas y Navegación	30
5.4. Guards e Interceptores	31

5.4.1. AuthGuard	31
5.4.2. RoleGuard	31
5.4.3. Interceptor de Autenticación	32
6. Capítulo VI: Manual de Funcionalidades	34
6.1. Autenticación	34
6.1.1. Inicio de Sesión	34
6.1.2. Registro de Usuario	34
6.2. Gestión de Productos	35
6.2.1. Listado de Productos (Menú)	35
6.2.2. Crear Producto (Admin)	35
6.2.3. Editar y Eliminar Producto	36
6.3. Gestión de Categorías	36
6.4. Procesamiento de Pedidos	36
6.4.1. Carrito de Compras	36
6.4.2. Finalizar Compra	37
6.4.3. Seguimiento de Pedidos	37
6.5. Panel Administrativo	38
6.5.1. Dashboard Principal	38
6.5.2. Gestión de Pedidos (Admin)	38
6.5.3. Gestión de Promociones y Mensajes	38
6.6. Administración de Usuarios	39
7. Capítulo VII: Funcionalidades Avanzadas	40
7.1. Contenerización con Docker	40
7.1.1. Dockerfile del Backend	40
7.1.2. Dockerfile del Frontend (Desarrollo)	40
7.1.3. Orquestación con Docker Compose	41
7.2. Despliegue en la Nube	41
7.2.1. Backend en Render	41
7.2.2. Frontend en Vercel	42
7.2.3. Base de Datos en Supabase	42
7.3. Seguridad	42
7.3.1. Autenticación JWT	42
7.3.2. Protección contra CSRF	42
7.3.3. Validación de Entradas	42
7.4. Escalabilidad	42
8. Capítulo VIII: Resultados y Pruebas	43
8.1. Validación de Persistencia (Supabase)	43
8.2. Validación del Frontend	44
8.3. Pruebas de API	44
8.4. Rendimiento	45
9. Capítulo IX: Conclusiones	46
9.1. Conclusiones	46
9.2. Recomendaciones	46
9.3. Trabajos Futuros	46
Bibliografía	47
Anexos	48

Índice de figuras

1.	Diagrama de arquitectura general de Cafe-Sys.	12
2.	Diagrama Entidad-Relación Conceptual de Cafe-Sys.	16
3.	Diagrama Entidad-Relación Físico de Cafe-Sys.	17
4.	Interfaz de inicio de sesión de Cafe-Sys.	34
5.	Interfaz de registro de usuario.	35
6.	Catálogo público de productos ("Nuestro Menú").	35
7.	Formulario de gestión de productos.	36
8.	Interfaz del carrito de compras ("Tu Cesta").	37
9.	Pantalla de confirmación tras finalizar la compra ("¡Pedido realizado con éxito!").	37
10.	Sección "Mis Pedidos" con el historial de órdenes del cliente.	38
11.	Panel de control del administrador.	38
12.	Tabla física <i>orders</i> en Supabase, mostrando estados de orden (<i>ready</i> , <i>assigned</i> , etc.), totales y notas tras peticiones POST exitosas.	43
13.	Tabla física <i>order_details</i> en Supabase, mostrando cantidades, precios unitarios y subtotales, evidenciando la integridad referencial con la tabla <i>orders</i>	43
14.	Sección de "Órdenes" en el frontend desplegado en Vercel, utilizada por conductores y empleados para gestionar el flujo de preparación y entrega de pedidos mediante las acciones <i>advance_status</i> y <i>archive</i>	44
15.	Panel de administración de Django en producción, mostrando la lista y gestión del catálogo de productos (<i>Products</i>).	44

Índice de cuadros

1.	Resumen del stack tecnológico de Cafe-Sys.	11
2.	Componentes de la capa de persistencia de datos.	14
3.	Estructura del modelo Role.	18
4.	Estructura del modelo User.	18
5.	Estructura del modelo Category.	18
6.	Estructura del modelo Product.	19
7.	Estructura del modelo Order.	19
8.	Estructura del modelo OrderDetail.	19
9.	Estructura del modelo Promotions.	20
10.	Catálogo completo de endpoints de la API REST.	24
11.	Matriz de roles y permisos efectivamente implementada en el sistema.	25
12.	Organización de componentes en Angular.	27
13.	Métricas de rendimiento del sistema.	45

1. Capítulo I: Introducción

1.1. Descripción del Problema

Las cafeterías universitarias enfrentan diariamente desafíos operativos significativos derivados de la ausencia de sistemas de gestión automatizados. Los procesos manuales de registro de pedidos, control de inventario y coordinación de entregas generan errores humanos, tiempos de espera prolongados y pérdida de información crítica para la toma de decisiones.

El presente proyecto, denominado *Cafe-Sys*, surge como respuesta a esta problemática, proponiendo una solución tecnológica integral que digitaliza y optimiza cada etapa del flujo operativo de una cafetería, desde la recepción del pedido hasta la entrega final al cliente.

1.2. Objetivos

1.2.1. Objetivo General

Desarrollar una aplicación web full-stack para la gestión integral de una cafetería universitaria, que permita automatizar los procesos de pedidos, inventario, logística de entregas y generación de reportes estadísticos, utilizando tecnologías modernas de desarrollo web.

1.2.2. Objetivos Específicos

1. Diseñar e implementar una base de datos relacional en PostgreSQL que modele las entidades del negocio (productos, categorías, pedidos, usuarios, conductores, vehículos) con integridad referencial.
2. Desarrollar una API RESTful con Django REST Framework que exponga endpoints seguros para la manipulación de datos, implementando autenticación JWT y permisos diferenciados.
3. Construir una interfaz de usuario moderna y responsiva con Angular v21, implementando arquitectura modular, lazy loading y programación reactiva con RxJS.
4. Implementar un módulo de logística que permita la asignación de repartidores a pedidos y el seguimiento de estados de entrega en tiempo real.
5. Desplegar el sistema en infraestructura cloud (Render para backend, Vercel para frontend, Supabase para base de datos) utilizando contenedores Docker.
6. Documentar el sistema mediante un informe técnico que consolide las funcionalidades por categorías (backend, frontend, base de datos) con diagramas, tablas y capturas de pantalla.

1.3. Alcance

1.3.1. Qué hace el sistema

- Gestión completa de productos y categorías (CRUD).
- Registro y seguimiento de pedidos con detalles anidados.
- Administración de usuarios con roles diferenciados (administrador, cliente, repartidor).
- Módulo de logística con asignación de conductores y vehículos a pedidos.
- Panel de reportes estadísticos con gráficos de ventas e inventario.
- Autenticación segura mediante JWT con control de sesiones.
- Interfaz responsiva adaptable a dispositivos móviles y de escritorio.

1.3.2. Qué no hace el sistema

- No implementa pasarela de pagos en línea (solo registra el pedido).
- No incluye geolocalización GPS en tiempo real para repartidores.
- No gestiona múltiples sucursales o franquicias.
- No incluye módulo de facturación electrónica con SUNAT.

1.4. Tecnologías Utilizadas

Tecnología	Versión	Uso en el Proyecto
Angular	v21.1.0	Frontend, interfaz de usuario SPA
Django	6.0.5	Backend, ORM, panel de administración
Django REST Framework	3.17.1	API REST, serializadores, permisos
PostgreSQL	16.x	Base de datos relacional
Supabase	–	Hosting de base de datos PostgreSQL
Docker	27.x	Contenerización del entorno
Render	–	Despliegue del backend
Vercel	–	Despliegue del frontend
Simple JWT	5.3.1	Autenticación con tokens JWT
RxJS	7.8.0	Programación reactiva en Angular
TypeScript	5.9.2	Tipado estático en el frontend
Gunicorn	23.0.0	Servidor WSGI en producción
Whitenoise	6.9.0	Servicio de archivos estáticos

Cuadro 1: Resumen del stack tecnológico de Cafe-Sys.

2. Capítulo II: Arquitectura del Sistema

2.1. Arquitectura General

El sistema *Cafe-Sys* sigue una arquitectura de tres capas (Three-Tier Architecture), donde cada capa tiene responsabilidades bien definidas y se comunica mediante protocolos estandarizados. La Figura 1 ilustra el flujo de datos entre los componentes principales.

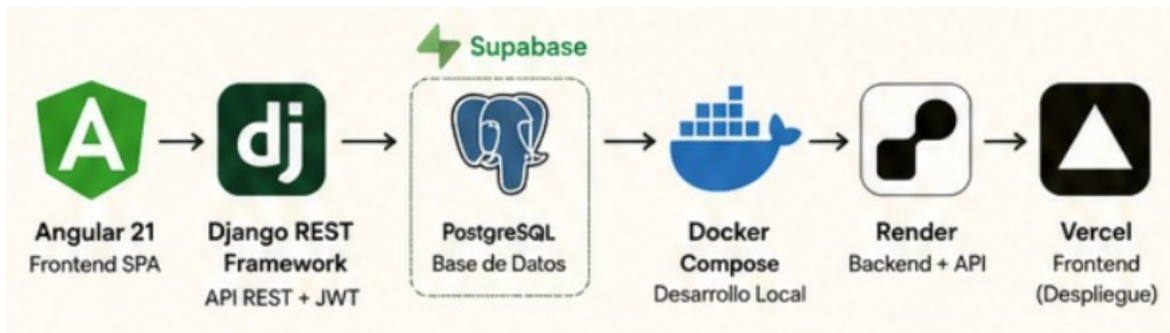


Figura 1: Diagrama de arquitectura general de Cafe-Sys.

El flujo de comunicación opera de la siguiente manera:

1. El **frontend** (Angular) realiza peticiones HTTP al backend mediante el servicio `HttpClient`.
2. El **backend** (Django + DRF) recibe la petición, valida permisos JWT, ejecuta la lógica de negocio y consulta la base de datos.
3. La **base de datos** (PostgreSQL en Supabase) almacena y recupera los datos según las consultas del ORM de Django.
4. La respuesta JSON viaja en sentido inverso hasta renderizarse en la interfaz del usuario.

2.2. Arquitectura Backend

El backend está organizado siguiendo las convenciones de Django, con una estructura modular que facilita el mantenimiento y la escalabilidad.

Listing 1: Estructura de carpetas del backend

```
backend/
|
|-- apps/
|   |-- core/                # Aplicacion principal
|   |   |-- migrations/      # Historial de migraciones
|   |   |-- models/          # Definicion de tablas
|   |   |-- admin.py          # Panel de administracion
|   |   |-- apps.py           # Configuracion de la app
|   |   |-- serializers.py    # Conversion modelo -> JSON
|   |   |-- urls.py           # Enrutamiento de la API
|   |   |-- views.py          # Logica de controladores
|   |
|   |-- cafesys/              # Configuracion global
|   |   |-- settings.py       # Ajustes y CORS
|   |   |-- urls.py           # Enrutador raiz
|   |   |-- asgi.py
```

```
| |-- wsgi.py
|
|-- venv/                # Entorno virtual
|-- .env                 # Variables de entorno
|-- Dockerfile           # Contenedor backend
|-- manage.py
|-- requirements.txt
```

Descripción de componentes clave:

- **apps/core/models/**: Contiene las definiciones de todas las entidades del sistema (Product, Order, User, Driver, Vehicle, Category).
- **serializers.py**: Transforma instancias de modelos a representaciones JSON y viceversa, incluyendo serializadores anidados para relaciones complejas.
- **views.py**: Implementa los ViewSets de DRF que manejan las operaciones CRUD sobre cada recurso.
- **permissions.py**: Define políticas de acceso personalizadas, como lectura pública y escritura autenticada.

2.3. Arquitectura Frontend

El frontend sigue la estructura recomendada por Angular para aplicaciones empresariales, con separación por módulos funcionales.

Listing 2: Estructura de carpetas del frontend

```
frontend/
| | | public/                # Archivos estticos (favicon, imgenes, assets)
| | | | | favicon.ico
| | | | | src/
| | | | | | | app/
| | | | | | | | | guards/      # Proteccin de rutas
| | | | | | | | | | | auth.guard.ts
| | | | | | | | | | | role.guard.ts
| | | | | | | | | | | interceptors/ # Interceptores HTTP
| | | | | | | | | | | | | auth.interceptor.ts
| | | | | | | | | | | pages/      # Vistas y pginas de la aplicacin
| | | | | | | | | | | | | admin/   # Panel de administracin
| | | | | | | | | | | | | | | mensajes/
| | | | | | | | | | | | | | | ordenes/
| | | | | | | | | | | | | | | users/
| | | | | | | | | | | | | | | | cart/      # Carrito de compras
| | | | | | | | | | | | | | | | categories/ # Catlogo de categoras
| | | | | | | | | | | | | | | | contact/   # Formulario de contacto
| | | | | | | | | | | | | | | | dashboard/ # Panel de control
| | | | | | | | | | | | | | | | home/      # Pgina de inicio
| | | | | | | | | | | | | | | | layouts/   # Estructuras comunes (Navbar, Sidebar)
| | | | | | | | | | | | | | | | location/  # Ubicacin de sucursales
| | | | | | | | | | | | | | | | login/     # Autenticacin
| | | | | | | | | | | | | | | | menu/      # Carta / Men
| | | | | | | | | | | | | | | | mis-pedidos/ # Historial de pedidos del usuario
| | | | | | | | | | | | | | | | products/  # Detalle y gestin de productos
| | | | | | | | | | | | | | | | register/  # Registro de usuarios
| | | | | | | | | | | | | | | | services/  # Servicios y lgica de negocio
| | | | | | | | | | | | | | | | api-config.ts
| | | | | | | | | | | | | | | | api.spec.ts # Pruebas unitarias del servicio API
| | | | | | | | | | | | | | | | api.ts
```

```

| | | | auth.service.ts
| | | | cart.service.ts
| | | | app.config.ts # Configuración global
| | | | app.routes.ts # Enrutamiento
| | | | app.ts # Componente raíz
| | | | index.html
| | | | main.ts # Punto de entrada
| | | | styles.css # Estilos globales
| | angular.json
| | Dockerfile
| | package.json
| | tsconfig.json
| | vercel.json

```

Descripción de componentes clave:

- **pages/**: Aloja los módulos funcionales y las vistas de la aplicación, los cuales se cargan bajo demanda (*lazy loading*) para optimizar el rendimiento inicial.
- **services/**: Centraliza la comunicación HTTP con el backend mediante `HttpClient` y `Observables` de `RxJS`.
- **guards/**: Protege las rutas según el estado de autenticación y los roles del usuario.
- **interceptors/**: Inyecta automáticamente el token JWT en cada petición y maneja los errores globales.

2.4. Arquitectura de Base de Datos

La base de datos está diseñada siguiendo el modelo relacional con normalización hasta la tercera forma normal (3NF). La conexión con Django se realiza mediante el adaptador `psycopg2-binary` y las credenciales se gestionan mediante variables de entorno.

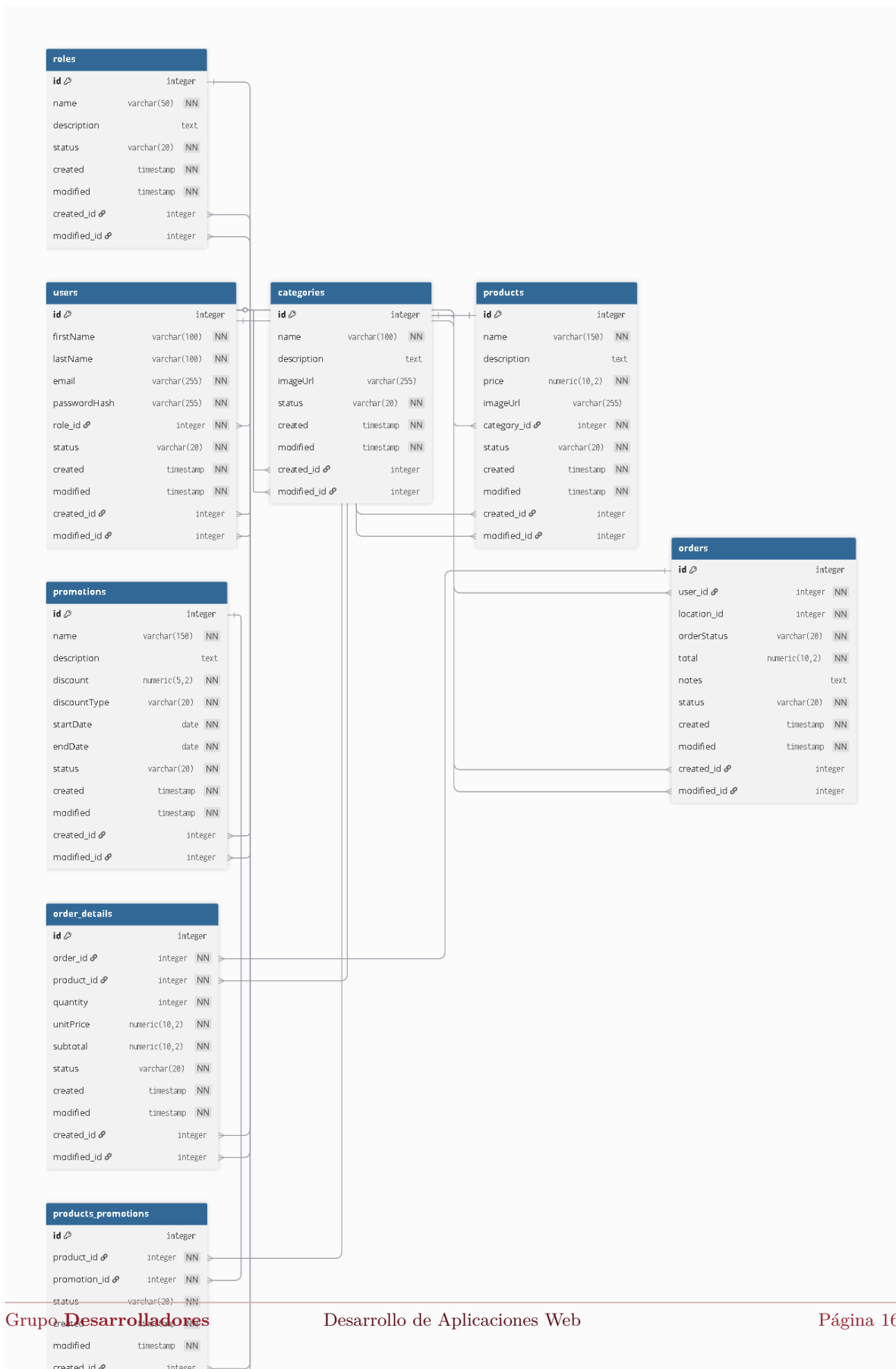
Componente	Tecnología	Descripción
Motor de BD	PostgreSQL 16	Base de datos relacional robusta
Hosting	Supabase	Plataforma cloud con PostgreSQL administrado
ORM	Django ORM	Mapeo objeto-relacional integrado
Driver	psycopg2-binary	Adaptador Python para PostgreSQL
Migrations	Django Migrations	Control de versiones del esquema

Cuadro 2: Componentes de la capa de persistencia de datos.

3. Capítulo III: Base de Datos

3.1. Diseño Conceptual (DER Conceptual)

El Diagrama Entidad-Relación Conceptual (Figura 2) representa las entidades del negocio y sus relaciones sin detallar tipos de datos físicos. Este modelo fue elaborado en dbdiagram.io y sirve para comunicar las reglas de negocio a stakeholders no técnicos.



Entidades principales y relaciones:

- **User:** Almacena información de clientes y administradores. Relación 1:N con Order.
- **Category:** Clasificación de productos. Relación 1:N con Product.
- **Product:** Catálogo de productos de la cafetería. Relación 1:N con OrderDetail.
- **Order:** Registro de pedidos. Relación 1:N con OrderDetail y 1:1 con Delivery.
- **OrderDetail:** Líneas de detalle de cada pedido (producto, cantidad, subtotal).
- **Driver:** Información de repartidores. Relación 1:N con Delivery.
- **Vehicle:** Vehículos asignados a repartidores. Relación 1:1 con Driver.
- **Delivery:** Registro de entregas con estado y asignación de conductor.

3.2. Diseño Lógico y Físico (DER Detallado)

El Diagrama Entidad-Relación Físico (Figura 3) especifica los tipos de datos, claves primarias, foráneas e índices. Fue generado con Graphviz para representar el esquema real de PostgreSQL.

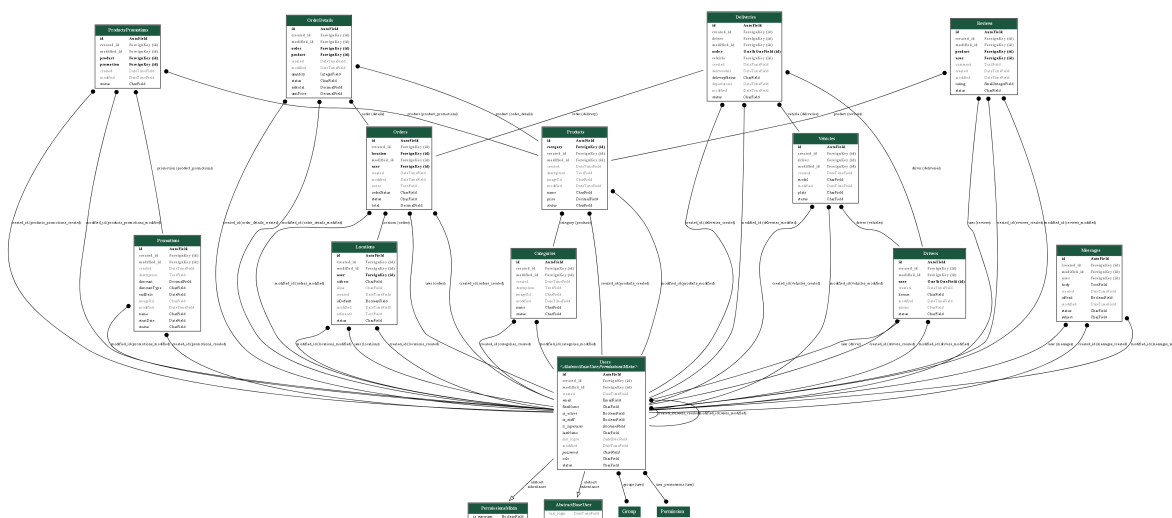


Figura 3: Diagrama Entidad-Relación Físico de Cafe-Sys.

3.3. Descripción de Modelos

A continuación se describe cada modelo implementado en el backend, detallando su estructura física real, tipos de datos y restricciones basadas en el esquema de la base de datos de Cafe-Sys.

3.3.1. Modelo Role (Rol)

Define los niveles de acceso y permisos dentro del sistema (ej. Administrador, Cliente).

Campo	Tipo	Descripción
id	integer (PK)	Identificador único secuencial
name	varchar(50) [NN]	Nombre del rol
description	text	Detalles sobre las capacidades del rol
status	varchar(20) [NN]	Estado operativo del rol
created	timestamp [NN]	Fecha y hora de registro
modified	timestamp [NN]	Última actualización del registro

Cuadro 3: Estructura del modelo Role.

3.3.2. Modelo User (Usuario)

Almacena las credenciales y la información de perfil de todos los actores del sistema.

Campo	Tipo	Descripción
id	integer (PK)	Identificador único del usuario
firstName	varchar(100) [NN]	Nombre(s) del usuario
lastName	varchar(100) [NN]	Apellido(s) del usuario
email	varchar(255) [NN]	Correo electrónico (único)
passwordHash	varchar(255) [NN]	Contraseña encriptada
role_id	integer (FK)	Relación con el modelo Role
status	varchar(20) [NN]	Estado de la cuenta (ej. Activo)
created	timestamp [NN]	Fecha de registro
modified	timestamp [NN]	Última modificación

Cuadro 4: Estructura del modelo User.

3.3.3. Modelo Category (Categoría)

Permite agrupar y clasificar los productos del menú.

Campo	Tipo	Descripción
id	integer (PK)	Identificador de la categoría
name	varchar(100) [NN]	Nombre de la categoría
description	text	Breve descripción comercial
imageUrl	varchar(255)	Ruta o URL de la imagen del ícono
status	varchar(20) [NN]	Vigencia de la categoría
created	timestamp [NN]	Fecha de creación
modified	timestamp [NN]	Última modificación

Cuadro 5: Estructura del modelo Category.

3.3.4. Modelo Product (Producto)

Gestiona el catálogo de productos disponibles para la venta.

Campo	Tipo	Descripción
id	integer (PK)	Identificador único del producto
name	varchar(150) [NN]	Nombre del producto
description	text	Ingredientes o detalle del artículo
price	numeric(10,2) [NN]	Precio unitario de venta
imageUrl	varchar(255)	URL de la fotografía del producto
category_id	integer (FK)	Relación jerárquica con Category
status	varchar(20) [NN]	Disponibilidad en tienda
created	timestamp [NN]	Fecha de alta del producto
modified	timestamp [NN]	Último cambio en el catálogo

Cuadro 6: Estructura del modelo Product.

3.3.5. Modelo Order (Pedido)

Modela la cabecera de las transacciones de compra de la cafetería.

Campo	Tipo	Descripción
id	integer (PK)	Número de pedido único
user_id	integer (FK)	Comprador (vía el modelo User)
location_id	integer	Sucursal donde se procesa la orden
orderStatus	varchar(20) [NN]	Estado del ciclo de vida de la orden
total	numeric(10,2) [NN]	Costo total acumulado de la compra
notes	text	Indicaciones especiales de preparación
status	varchar(20) [NN]	Estado del registro
created	timestamp [NN]	Fecha y hora exacta de la compra
modified	timestamp [NN]	Actualización del estado del pedido

Cuadro 7: Estructura del modelo Order.

3.3.6. Modelo OrderDetail (Detalle de Pedido)

Desglosa los productos específicos y cantidades asociados a una orden.

Campo	Tipo	Descripción
id	integer (PK)	Identificador de línea de detalle
order_id	integer (FK)	Vinculación a la cabecera Order
product_id	integer (FK)	Producto adquirido
quantity	integer [NN]	Cantidad de unidades solicitadas
unitPrice	numeric(10,2) [NN]	Precio congelado al momento del pago
subtotal	numeric(10,2) [NN]	Cálculo de $quantity \times unitPrice$
status	varchar(20) [NN]	Estado de la línea

Cuadro 8: Estructura del modelo OrderDetail.

3.3.7. Modelo Promotions (Promociones)

Administra las campañas de descuentos y ofertas globales sobre los productos.

Campo	Tipo	Descripción
id	integer (PK)	Identificador de la promoción
name	varchar(150) [NN]	Nombre de la campaña de descuento
description	text	Condiciones de la oferta
discount	numeric(5,2) [NN]	Magnitud porcentual o fija del descuento
discountType	varchar(20) [NN]	Tipo de descuento (porcentaje, directo)
startDate	date [NN]	Fecha de inicio de vigencia
endDate	date [NN]	Fecha de caducidad
status	varchar(20) [NN]	Si la promoción está activa o pausada

Cuadro 9: Estructura del modelo Promotions.

4. Capítulo IV: Backend

4.1. Organización del Proyecto

El backend de *Cafe-Sys* está construido sobre Django 6.0.5 con Django REST Framework 3.17.1. La organización del código sigue el principio de responsabilidad única, separando la lógica de negocio en aplicaciones modulares dentro del directorio `apps/`.

Convención de nombres: Todos los endpoints de la API siguen el patrón `/api/<recurso>/` para mantener consistencia y facilitar la documentación automática.

4.2. Serializers

Los serializadores de DRF convierten instancias de modelos Django a JSON y validan los datos entrantes. Se implementaron serializadores anidados para reducir el número de peticiones HTTP, utilizando versiones reducidas de los modelos relacionados cuando no se requiere el detalle completo.

4.2.1. Serializador de Productos

Listing 3: ProductsSerializer en serializers.py

```
class ProductsSerializer(serializers.ModelSerializer):
    # Mostrar nombre de la categoria en lectura (sin reemplazar el FK para escritura)
    category_name = serializers.CharField(
        source='category.name', read_only=True
    )

    class Meta:
        model = Products
        fields = (
            'id', 'name', 'description', 'price',
            'imageUrl', 'category', 'category_name',
            'status', 'created', 'modified'
        )
        read_only_fields = ('id', 'category_name', 'created', 'modified')
```

4.2.2. Serializadores Anidados para Pedidos

Para evitar sobrecargar la respuesta de un pedido con todos los campos de un producto, se definió un serializer reducido exclusivo para el contexto de detalle de pedido.

Listing 4: Serializadores anidados para Orders

```
class ProductDetailNestedSerializer(serializers.ModelSerializer):
    class Meta:
        model = Products
        fields = ('id', 'name', 'price', 'imageUrl', 'status')

class OrderDetailsNestedSerializer(serializers.ModelSerializer):
    product_detail = ProductDetailNestedSerializer(
        source='product', read_only=True
    )

    class Meta:
        model = OrderDetails
        fields = (
```

```
        'id', 'product', 'product_detail',
        'quantity', 'unitPrice', 'subtotal', 'status'
    )
    read_only_fields = ('id', 'subtotal', 'product_detail')

class OrderDetailWriteSerializer(serializers.Serializer):
    product = serializers.IntegerField()
    quantity = serializers.IntegerField(min_value=1)
    unitPrice = serializers.DecimalField(
        max_digits=10, decimal_places=2, min_value=0.01
    )

class OrdersSerializer(serializers.ModelSerializer):
    details = OrderDetailsNestedSerializer(many=True, read_only=True)
    user_detail = UsersSerializer(source='user', read_only=True)
    # Campos de entrada para crear detalles inline
    details_data = OrderDetailWriteSerializer(many=True, write_only=True, required=False)

class Meta:
    model = Orders
    fields = (
        'id', 'user', 'user_detail', 'location', 'orderStatus',
        'total', 'notes', 'details', 'details_data',
        'status', 'created', 'modified'
    )
    read_only_fields = ('id', 'details', 'user_detail', 'created', 'modified')

def create(self, validated_data):
    details_data = validated_data.pop('details_data', [])

    # El total no se confía al cliente: se recalcula a partir de los detalles
    computed_total = Decimal('0')
    for detail in details_data:
        qty = Decimal(str(detail['quantity']))
        unit_price = Decimal(str(detail['unitPrice']))
        subtotal = (qty * unit_price).quantize(Decimal('0.01'))
        computed_total += subtotal

    validated_data['total'] = computed_total.quantize(Decimal('0.01'))
    order = Orders.objects.create(**validated_data)

    for detail in details_data:
        qty = detail['quantity']
        unit_price = detail['unitPrice']
        subtotal = (Decimal(str(qty)) * Decimal(str(unit_price))).quantize(Decimal('0.01'))
        OrderDetails.objects.create(
            order=order,
            product_id=detail['product'],
            quantity=qty,
            unitPrice=unit_price,
            subtotal=subtotal
        )
    return order
```

4.3. Views y Controladores

Los ViewSets de DRF encapsulan la lógica CRUD para cada recurso, implementando permisos diferenciados según la acción y acciones personalizadas (`@action`) para transiciones de estado.

4.3.1. Permisos Diferenciados y Acciones Personalizadas

Listing 5: OrdersViewSet con permisos y transiciones de estado

```
class OrdersViewSet(viewsets.ModelViewSet):
    permission_classes = [IsAuthenticated]
    queryset = Orders.objects.select_related(
        'user', 'location'
    ).prefetch_related(
        'details', 'details__product'
    ).all()
    serializer_class = OrdersSerializer

    def get_permissions(self):
        # GET (list/retrieve) publico, resto requiere autenticacion
        if self.action in ('list', 'retrieve'):
            return [AllowAny()]
        return [IsAuthenticated()]

    @action(detail=True, methods=['post'], permission_classes=[IsAuthenticated])
    def advance_status(self, request, pk=None):
        # Avanza el pedido al siguiente estado del flujo:
        # pending -> assigned -> preparing -> ready -> delivered
        order = self.get_object()
        ...

    @action(detail=True, methods=['post'], permission_classes=[IsAuthenticated])
    def cancel(self, request, pk=None):
        # Solo el dueño del pedido puede cancelarlo, y solo si esta 'pending'
        order = self.get_object()
        ...
```

4.4. Endpoints de la API

La siguiente tabla consolida todos los endpoints principales de la API REST, según están registrados en el `DefaultRouter` de `urls.py`:

Endpoint	Método	Autenticación	Función
/api/token/	POST	No	Obtener tokens JWT (login)
/api/token/refresh/	POST	No	Refrescar token de acceso
/api/users/	GET	Sí	Listar usuarios
/api/users/	POST	No	Registrar nuevo usuario
/api/users/{id}/	GET/PUT/DELETE	Sí	Ver, actualizar o eliminar usuario
/api/categories/	GET	No	Listar categorías
/api/categories/	POST/PUT/DELETE	Sí	Gestionar categorías
/api/products/	GET	No	Listar / ver detalle de producto
/api/products/	POST/PUT/DELETE	Sí	Crear, actualizar o eliminar producto
/api/promotions/	GET	No	Listar promociones activas
/api/promotions/	POST/PUT/DELETE	Sí	Gestionar promociones
/api/products-promotions/	GET/POST/DELETE	Según config. global	Relación producto-promoción
/api/locations/	GET/POST/DELETE	Sí	Direcciones de usuarios
/api/orders/	GET	No	Listar / ver detalle de pedido
/api/orders/	POST/PUT	Sí	Crear o actualizar pedido
/api/orders/{id}/advance-status/	POST	Sí	Avanzar estado del pedido
/api/orders/{id}/cancel/	POST	Sí (dueño)	Cancelar pedido pendiente
/api/orders/{id}/archive/	POST	Sí (staff/empleador)	Archivar pedido
/api/order-details/	GET	No	Listar / ver detalle de línea de pedido
/api/order-details/	POST/PUT/DELETE	Sí	Gestionar líneas de pedido
/api/drivers/	GET/POST/DELETE	Sí	Gestionar conductores
/api/vehicles/	GET/POST/DELETE	Sí	Gestionar vehículos
/api/deliveries/	GET/POST/DELETE	Sí	Gestionar entregas
/api/reviews/	GET	No	Listar reseñas
/api/reviews/	POST/PUT/DELETE	Sí	Gestionar reseñas
/api/messages/	POST	No	Formulario de contacto público
/api/messages/	GET/PUT/DELETE	Sí	Gestionar mensajes

Cuadro 10: Catálogo completo de endpoints de la API REST.

4.5. Autenticación y Seguridad

4.5.1. JSON Web Tokens (JWT)

La autenticación se implementa mediante `django-rest-framework-simplejwt`, que proporciona endpoints para obtener y refrescar tokens.

Listing 6: Configuración JWT en settings.py

```
REST_FRAMEWORK = {
    'DEFAULT_AUTHENTICATION_CLASSES': [
        'rest_framework_simplejwt.authentication.JWTAuthentication',
    ]
}
```



```

    ],
    'DEFAULT_PERMISSION_CLASSES': [
        'rest_framework.permissions.IsAuthenticated',
    ],
}
SIMPLE_JWT = {
    'ACCESS_TOKEN_LIFETIME': timedelta(minutes=60),
    'REFRESH_TOKEN_LIFETIME': timedelta(days=7),
    'ROTATE_REFRESH_TOKENS': True,
    'BLACKLIST_AFTER_ROTATION': True,
}

```

Cabe notar que `DEFAULT_PERMISSION_CLASSES` exige autenticación por defecto en toda la API; los `ViewSet`s individuales sobrescriben este comportamiento cuando necesitan acceso público (por ejemplo, `AllowAny` en lectura de productos o `IsAuthenticatedOrReadOnly` en categorías y promociones).

4.5.2. Roles y Permisos

A diferencia de un esquema de permisos por rol totalmente granular, el sistema actual combina permisos generales de DRF (`IsAuthenticated`, `IsAuthenticatedOrReadOnly`, `AllowAny`) con verificaciones explícitas de rol únicamente en operaciones críticas del flujo de pedidos.

Rol	Permisos efectivos	Descripción
Staff / Empleado (<code>is_staff</code> o <code>role</code> en <code>{Employee, Driver}</code>)	Avanzar y archivar pedidos	Puede ejecutar <code>advance_status</code> y <code>archive</code> sobre cualquier pedido, moviéndolo por el flujo <i>pending</i> → <i>assigned</i> → <i>preparing</i> → <i>ready</i> → <i>delivered</i>
Cliente (dueño del pedido)	Crear pedidos, cancelar los propios	Cualquier usuario autenticado puede crear pedidos; solo puede cancelar un pedido si es el dueño (<code>order.user_id == user.id</code>) y este está en estado <code>pending</code>
Usuario autenticado (genérico)	CRUD en productos, categorías, promociones, ubicaciones, vehículos, etc.	El sistema no restringe estas operaciones a un rol de <code>.administrador.específico</code> : cualquier usuario autenticado con <code>IsAuthenticated</code> o <code>IsAuthenticatedOrReadOnly</code> puede realizarlas
Anónimo	Lectura pública	Puede listar y ver detalle de productos, categorías, promociones, pedidos y reseñas; puede registrarse (<code>POST /api/users/</code>) y enviar mensajes de contacto (<code>POST /api/messages/</code>)

Cuadro 11: Matriz de roles y permisos efectivamente implementada en el sistema.

4.5.3. Configuración CORS

Listing 7: Configuración CORS para comunicación frontend-backend

```

CORS_ALLOWED_ORIGINS = [
    'https://cafe-sys.vercel.app',
    'http://localhost:4200',
]

```

```
'http://localhost:8082',  
]  
  
CORS_ALLOW_CREDENTIALS = True
```

5. Capítulo V: Frontend

5.1. Arquitectura de Angular

El frontend de *Cafe-Sys* está desarrollado en Angular v21 utilizando **Standalone Components**, una arquitectura moderna que elimina la necesidad de módulos `NgModule` y simplifica la estructura del código. La carga de páginas se realiza mediante *lazy loading* con `loadComponent`.

5.1.1. Estructura de Componentes

La aplicación se organiza en dos zonas diferenciadas, cada una con su propio layout, más un conjunto de páginas de autenticación sin layout:

Tipo	Ubicación	Ejemplos
Layout Cliente	pages/layouts/app-layout/	ClientLayoutComponent
Layout Admin	pages/layouts/admin-layout/	AdminLayoutComponent
Páginas Cliente	pages/¡nombre!_/	HomeComponent, MenuComponent, CartComponent, LocationComponent, MisPedidosComponent, ContactComponent
Páginas Admin	pages/admin/¡modulo!_/	OrdenesComponent, UsersComponent, MensajesComponent
Páginas Admin (raíz)	pages/¡nombre!_/	DashboardComponent, ProductsComponent, CategoriesComponent
Autenticación	pages/login/, pages/register/	LoginComponent, RegisterComponent (sin layout)

Cuadro 12: Organización de componentes en Angular.

5.2. Servicios

Los servicios centralizan la lógica de comunicación con el backend y el estado de la aplicación.

5.2.1. ApiService

El `ApiService` expone un método por recurso de la API REST, tipando cada respuesta con interfaces que reflejan exactamente el formato que devuelve Django (por ejemplo, campos en `camelCase` como `imageUrl` o `unitPrice`, y `DecimalField` serializado como `string`).

Listing 8: Interfaces principales devueltas por Django

```
export interface Producto {
  id: number;
  name: string;
  description: string | null;
  price: string; // DRF serializa DecimalField como string
  imageUrl: string | null;
  category: number;
  category_name: string;
  status: string;
}
```

```
export interface Order {
  id: number;
  user: number;
  user_detail: UserDetail;
  location: number;
  orderStatus: string;
  total: string;
  notes: string | null;
  details: OrderDetailNested[];
  status: string;
  created: string;
  modified: string;
}

export interface OrderDetailNested {
  id: number;
  product: number;
  product_detail: ProductDetail;
  quantity: number;
  unitPrice: string;
  subtotal: string;
  status: string;
}
```

Listing 9: Metodos principales de ApiService

```
@Injectable({ providedIn: 'root' })
export class ApiService {

  private readonly API_URL = getApiUrl();
  private readonly TIMEOUT = 10_000;

  constructor(private readonly http: HttpClient) {}

  // === PRODUCTOS ===
  getProducts(): Observable<Producto[]> {
    return this.http.get<Producto[]>({
      url: `${this.API_URL}/products/`,
    }).pipe(timeout(this.TIMEOUT));
  }

  // === ORDENES ===
  getOrders(): Observable<Order[]> {
    return this.http.get<Order[]>({
      url: `${this.API_URL}/orders/`,
    }).pipe(timeout(this.TIMEOUT));
  }

  createOrder(orderData: NewOrder): Observable<Order> {
    return this.http.post<Order>({
      url: `${this.API_URL}/orders/`, orderData
    }).pipe(timeout(this.TIMEOUT));
  }

  advanceOrderStatus(id: number): Observable<Order> {
```

```

        return this.http.post<Order>(
            `${this.API_URL}/orders/${id}/advance_status/`, {}
        ).pipe(timeout(this.TIMEOUT));
    }

    cancelOrder(id: number): Observable<Order> {
        return this.http.post<Order>(
            `${this.API_URL}/orders/${id}/cancel/`, {}
        ).pipe(timeout(this.TIMEOUT));
    }
}

```

Además de productos y órdenes, **ApiService** implementa métodos equivalentes para categorías, ubicaciones, detalles de orden, usuarios, conductores (*drivers*), entregas (*deliveries*), promociones, reseñas y mensajes, siguiendo el mismo patrón de inicio HTTP con `timeout`.

5.2.2. AuthService

Gestiona el estado de autenticación del usuario utilizando **signals** de Angular para reactividad. A diferencia de un enfoque con **localStorage**, los tokens se mantienen en signals privados en memoria y se replican en **sessionStorage** únicamente para sobrevivir recargas de página (más seguro que **localStorage**, ya que se borra al cerrar la pestaña).

Listing 10: Servicio de autenticación con signals

```

@Injectable({ providedIn: 'root' })
export class AuthService {

    private readonly API_URL = getApiUrl();

    private _accessToken = signal<string | null>(null);
    private _refreshToken = signal<string | null>(null);
    private _usuario = signal<UsuarioSesion | null>(null);

    readonly estaAutenticado = computed(() => this._accessToken() !== null);
    readonly usuario = computed(() => this._usuario());

    constructor(
        private http: HttpClient,
        private router: Router
    ) {
        this._restaurarSesion();
    }

    login(credenciales: LoginCredentials): Observable<UsuarioSesion> {
        return this.http.post<AuthTokens>(`${this.API_URL}/token/`, credenciales).pipe(
            tap(tokens => this._guardarTokens(tokens)),
            map(tokens => this._decodificarToken(tokens.access)),
            switchMap(payload => {
                if (!payload?.user_id) {
                    throw new Error('Token invalido: falta user_id');
                }
                return this.http.get<UsuarioSesion>(`${this.API_URL}/users/${payload.user_id}/`);
            }),
            tap(usuario => {
                this._usuario.set(usuario);
            })
        );
    }
}

```

```
        sessionStorage.setItem('cafesys_usuario', JSON.stringify(usuario));
    })
    );
}

logout(): void {
    this._accessToken.set(null);
    this._refreshToken.set(null);
    this._usuario.set(null);
    sessionStorage.clear();
    this.router.navigate(['/login']);
}
}
```

El método `login` no recibe directamente los datos del usuario desde el endpoint de token: decodifica el JWT para extraer el `user_id` y luego realiza una segunda petición a `/users/<id>/` para obtener el perfil completo.

5.3. Rutas y Navegación

El enrutamiento distingue una zona pública para clientes y una zona administrativa protegida, además de rutas de autenticación sin layout:

Listing 11: Configuración de rutas en `app.routes.ts`

```
export const routes: Routes = [
  { path: '', redirectTo: 'tienda', pathMatch: 'full' },

  // ZONA PUBLICA / CLIENTES
  {
    path: 'tienda',
    loadComponent: () => import('./pages/layouts/app-layout/app-layout')
      .then(m => m.ClientLayoutComponent),
    children: [
      { path: '', loadComponent: () => import('./pages/home/home')
        .then(m => m.HomeComponent) },
      { path: 'menu', loadComponent: () => import('./pages/menu/menu')
        .then(m => m.MenuComponent) },
      { path: 'carrito', loadComponent: () => import('./pages/cart/cart')
        .then(m => m.CartComponent) },
      { path: 'mis-pedidos', loadComponent: () => import('./pages/mis-pedidos/mis-pedidos')
        .then(m => m.MisPedidosComponent) },
    ]
  },

  // ZONA ADMINISTRATIVA
  {
    path: 'admin',
    loadComponent: () => import('./pages/layouts/admin-layout/admin-layout')
      .then(m => m.AdminLayoutComponent),
    canActivate: [AuthGuard, roleGuard],
    children: [
      { path: '', redirectTo: 'dashboard', pathMatch: 'full' },
      { path: 'dashboard', loadComponent: () => import('./pages/dashboard/dashboard')
        .then(m => m.DashboardComponent) },
      { path: 'productos', loadComponent: () => import('./pages/products/products')

```

```

        .then(m => m.ProductsComponent), data: { adminOnly: true } },
        { path: 'ordenes', loadComponent: () => import('./pages/admin/ordenes/ordenes')
          .then(m => m.OrdenesComponent) },
      ],
    },

    // ORDENES PUBLICAS (solo lectura, sin autenticacion)
    { path: 'ordenes', loadComponent: () => import('./pages/admin/ordenes/ordenes')
      .then(m => m.OrdenesComponent) },

    // AUTENTICACION (sin layout)
    { path: 'login', loadComponent: () => import('./pages/login/login')
      .then(m => m.LoginComponent) },
    { path: 'register', loadComponent: () => import('./pages/register/register')
      .then(m => m.RegisterComponent) },

    { path: '**', redirectTo: 'tienda' }
  ];

```

Nótese que el componente `OrdenesComponent` se reutiliza en dos rutas distintas: una pública bajo `/ordenes` y otra protegida bajo `/admin/ordenes`, reflejando que el backend permite lectura pública de pedidos (`AllowAnonymous` en `list/retrieve`) pero restringe su gestión a usuarios autenticados.

5.4. Guards e Interceptores

5.4.1. AuthGuard

Verifica que exista una sesión activa antes de permitir el acceso a rutas protegidas:

Listing 12: AuthGuard para proteccion de rutas

```

export const authGuard: CanActivateFn = () => {
  const authService = inject(AuthService);
  const router = inject(Router);

  if (authService.estaAutenticado()) {
    return true;
  }

  router.navigate(['/login']);
  return false;
};

```

5.4.2. RoleGuard

Además del guard de autenticación, un guard separado verifica el rol del usuario para decidir el acceso a la zona administrativa y a subrutas específicas dentro de ella:

Listing 13: RoleGuard con verificacion granular por ruta

```

export const roleGuard: CanActivateFn = (route, state) => {
  const authService = inject(AuthService);
  const router = inject(Router);
  const usuario = authService.usuario();

  if (!usuario) {
    return router.createUrlTree(['/login']);
  }
};

```

```

}

const allowedAdminLayoutRoles = ['Admin', 'Driver', 'Employee'];

if (!usuario.is_staff && !allowedAdminLayoutRoles.includes(usuario.role)) {
  return router.createUrlTree(['/tienda']);
}

const adminOnly = route.data?.['adminOnly'] as boolean | undefined;
const driverEmployeeOnly = route.data?.['driverEmployeeOnly'] as boolean | undefined;

if (adminOnly && usuario.role !== 'Admin') {
  return router.createUrlTree(['/admin/dashboard']);
}

if (driverEmployeeOnly && ![ 'Driver', 'Employee' ].includes(usuario.role)) {
  return router.createUrlTree(['/admin/dashboard']);
}

return true;
};

```

Este esquema permite, por ejemplo, que un usuario con rol **Driver** acceda al layout administrativo y a `/admin/ordenes`, pero sea redirigido fuera de rutas marcadas con `adminOnly: true` como `/admin/productos` o `/admin/usuarios`.

5.4.3. Interceptor de Autenticación

A diferencia de un interceptor funcional simple, el interceptor implementado es una clase que además intenta refrescar automáticamente el token de acceso ante una respuesta 401 antes de cerrar la sesión:

Listing 14: AuthInterceptor con reintento automatico

```

@Injectables()
export class AuthInterceptor implements HttpInterceptor {

  constructor(private authService: AuthService) {}

  intercept(req: HttpRequest<any>, next: HttpHandler): Observable<HttpEvent<any>> {
    const isAuthRequest = req.url.includes('/token/') || req.url.includes('/token/refresh/');
    const reqConToken = isAuthRequest ? req : this._agregarToken(req);

    return next.handle(reqConToken).pipe(
      catchError((error: HttpResponse) => {
        if (error.status === 401) {
          return this.authService.refreshAccessToken().pipe(
            switchMap(() => next.handle(this._agregarToken(req))),
            catchError((errorRefresh) => {
              this.authService.logout();
              return throwError(() => errorRefresh);
            })
          );
        }
        return throwError(() => error);
      })
    );
  }
}

```



```
}  
  
private _agregarToken(req: HttpRequest<any>): HttpRequest<any> {  
    const token = this.authService.getAccessToken();  
    if (!token) return req;  
  
    return req.clone({  
        setHeaders: { Authorization: 'Bearer ${token}' }  
    });  
}  
}
```

Este comportamiento evita cerrar la sesión del usuario ante la primera expiración del token de acceso: solo se ejecuta `logout()` si el propio refresco también falla.

6. Capítulo VI: Manual de Funcionalidades

Este capítulo documenta cada funcionalidad del sistema como un manual de usuario técnico, incluyendo descripción del flujo, capturas de pantalla y comportamiento esperado.

6.1. Autenticación

6.1.1. Inicio de Sesión

Descripción: El usuario ingresa su correo y contraseña en el formulario de login. El sistema valida las credenciales contra el backend y mantiene la sesión mediante signals en memoria, respaldados en `sessionStorage` para sobrevivir recargas de página.

Flujo:

1. El usuario accede a `/login`.
2. Completa el formulario con email y password.
3. El frontend envía POST a `/api/token/`.
4. El backend responde con los tokens `access` y `refresh`.
5. El frontend decodifica el JWT para obtener el `user_id`, solicita el perfil completo a `/api/users/<id>/` y redirige según el rol del usuario.

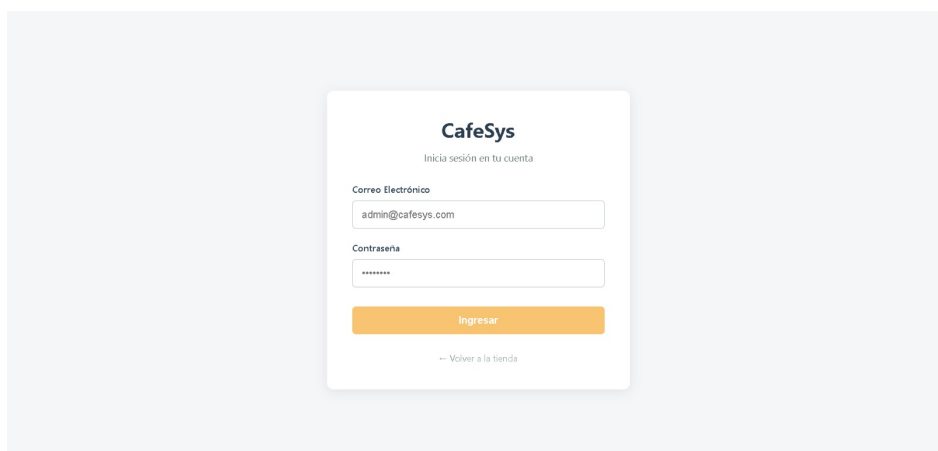


Figura 4: Interfaz de inicio de sesión de Cafe-Sys.

6.1.2. Registro de Usuario

Descripción: Los nuevos usuarios pueden crear una cuenta ingresando sus datos personales. El rol se asigna automáticamente como *Customer* en el backend; los roles administrativos (*Admin*, *Employee*, *Driver*) solo pueden asignarse desde el panel de Django Admin o mediante actualización directa de un usuario existente.

Flujo:

1. El usuario accede a `/register`.
2. Completa el formulario con nombre, apellido, correo y contraseña.
3. El frontend envía POST a `/api/users/`.
4. El backend crea el usuario con rol *Customer* por defecto y responde con los datos del perfil.

Únete a CafeSys
Crea tu cuenta y descubre el equilibrio perfecto.

Nombre: Ej. Diego
Apellido: Ej. Arce
Correo Electrónico: tu@correo.com
Contraseña: *****
Confirmar Contraseña: *****
Crear Cuenta

¿Ya tienes una cuenta? [Ingresa aquí](#)

[← Volver a la tienda](#)

Figura 5: Interfaz de registro de usuario.

6.2. Gestión de Productos

6.2.1. Listado de Productos (Menú)

Descripción: Vista pública que muestra el catálogo de productos disponibles con imagen, nombre, descripción, precio y categoría. Es accesible sin autenticación.

Funcionalidades:

- Búsqueda por nombre de producto.
- Filtrado por categoría.
- Ordenamiento por precio.

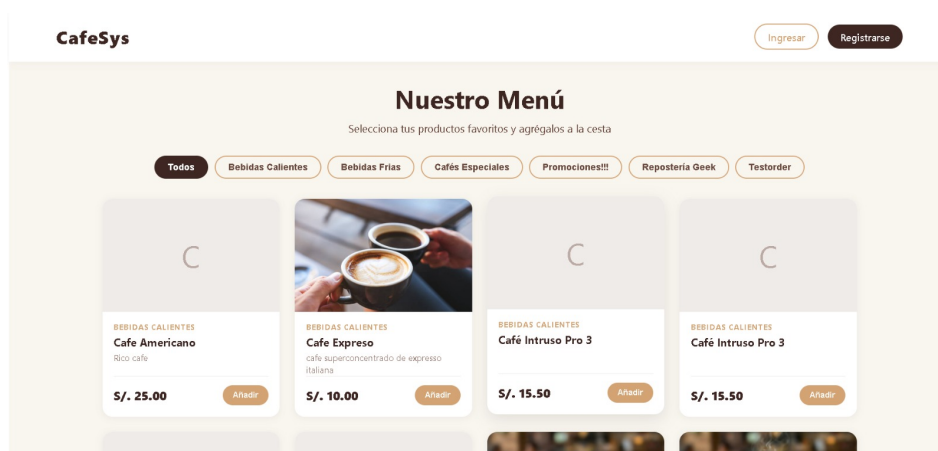


Figura 6: Catálogo público de productos ("Nuestro Menú").

6.2.2. Crear Producto (Admin)

Descripción: Desde `/admin/productos`, un usuario autenticado accede al formulario de creación donde ingresa nombre, descripción, precio, categoría e imagen del producto. Esta ruta está marcada como `adminOnly`, por lo que solo usuarios con rol `Admin` pueden acceder.

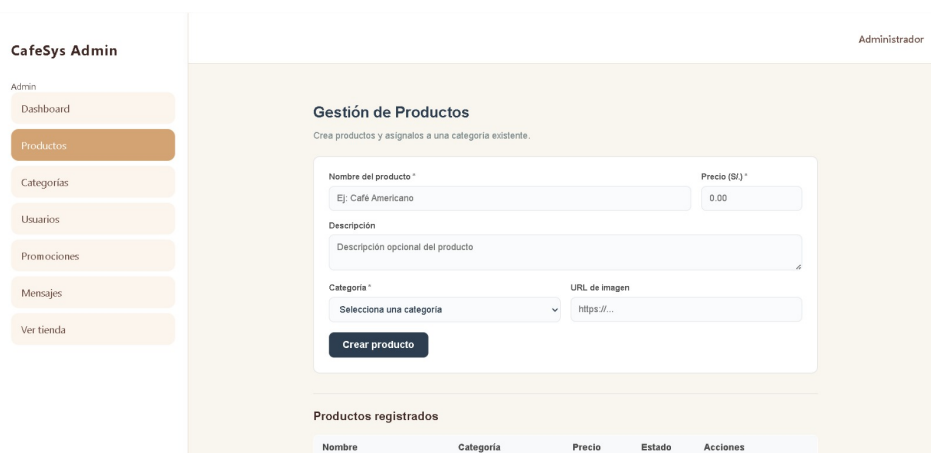


Figura 7: Formulario de gestión de productos.

6.2.3. Editar y Eliminar Producto

Descripción: Desde la misma vista de administración, el usuario con rol **Admin** puede editar cualquier campo del producto (PATCH /api/products/<id>/) o eliminarlo permanentemente del catálogo (DELETE /api/products/<id>/), lo que también elimina en cascada los **OrderDetails** asociados a ese producto.

6.3. Gestión de Categorías

Descripción: Módulo CRUD para la administración de categorías de productos, accesible en /admin/categorias y también marcado como adminOnly.

- **Agregar:** Formulario con nombre y descripción.
- **Editar:** Actualización parcial vía PATCH /api/categories/<id>/.
- **Eliminar:** Confirmación previa antes de DELETE /api/categories/<id>/.

6.4. Procesamiento de Pedidos

6.4.1. Carrito de Compras

Descripción: El cliente agrega productos al carrito desde /tienda/carrito, ajusta cantidades y visualiza el total acumulado. El carrito se mantiene en memoria durante la sesión del navegador.

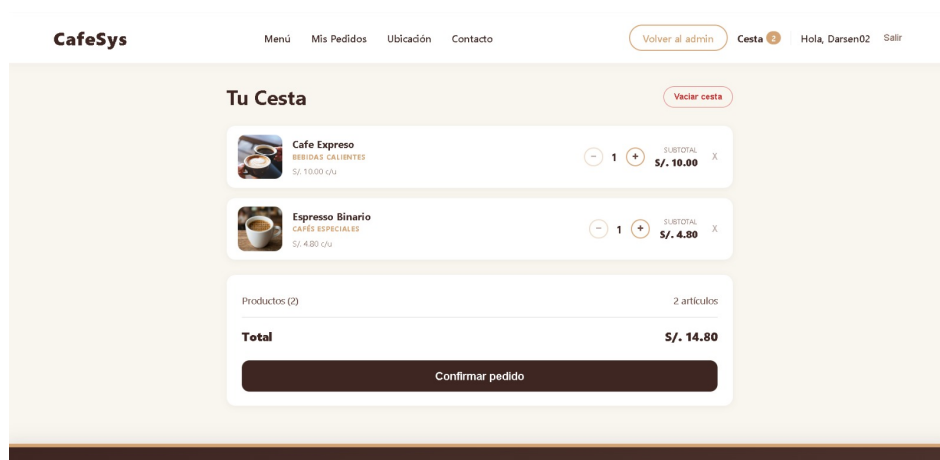


Figura 8: Interfaz del carrito de compras ("Tu Cesta").

6.4.2. Finalizar Compra

Descripción: El cliente confirma el pedido seleccionando una ubicación de entrega registrada y notas adicionales. El sistema calcula el total a partir de los productos y cantidades enviados, ignorando cualquier total propuesto por el cliente, y crea la orden junto con sus detalles en una sola transacción.

Flujo:

1. Validación de sesión de usuario (requiere autenticación).
2. Mapeo de productos del carrito al arreglo `details_data`.
3. Petición POST a `/api/orders/` con `location`, `notes` y `details_data`.
4. El backend recalcula el total y crea la orden junto con sus `OrderDetails` de forma atómica.

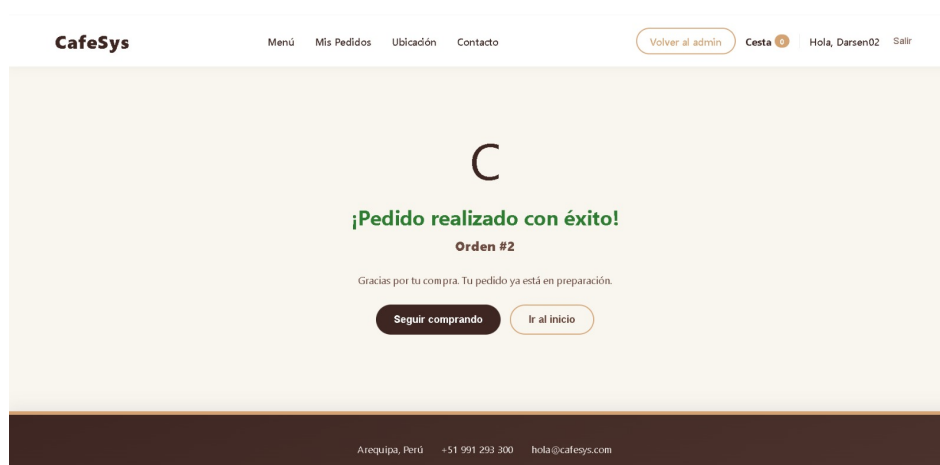


Figura 9: Pantalla de confirmación tras finalizar la compra ("¡Pedido realizado con éxito!").

6.4.3. Seguimiento de Pedidos

Descripción: Desde `/tienda/mis-pedidos`, los clientes consultan el historial y estado de sus pedidos: `pending`, `assigned`, `preparing`, `ready`, `delivered` o `cancelled`. Un cliente puede cancelar un pedido propio únicamente mientras esté en estado `pending`.

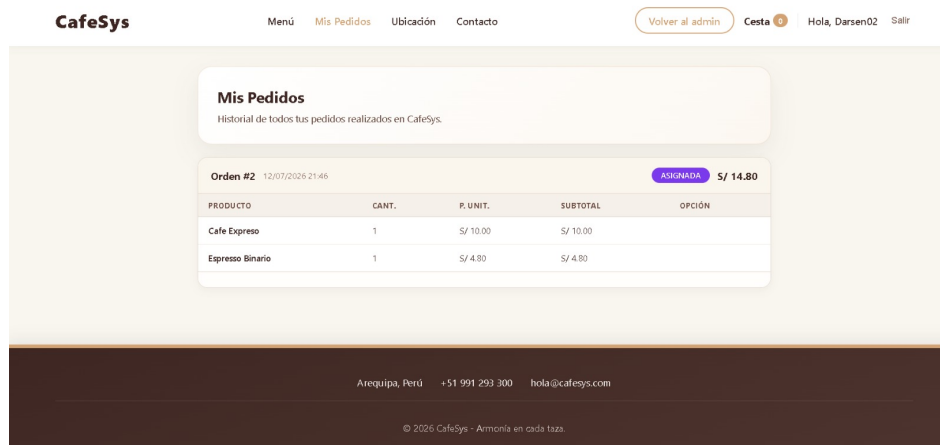


Figura 10: Sección "Mis Pedidos" con el historial de órdenes del cliente.

6.5. Panel Administrativo

6.5.1. Dashboard Principal

Descripción: Panel de control accesible en `/admin/dashboard`, protegido por `authGuard` y `roleGuard`, con acceso permitido a usuarios `Admin`, `Driver` y `Employee`.

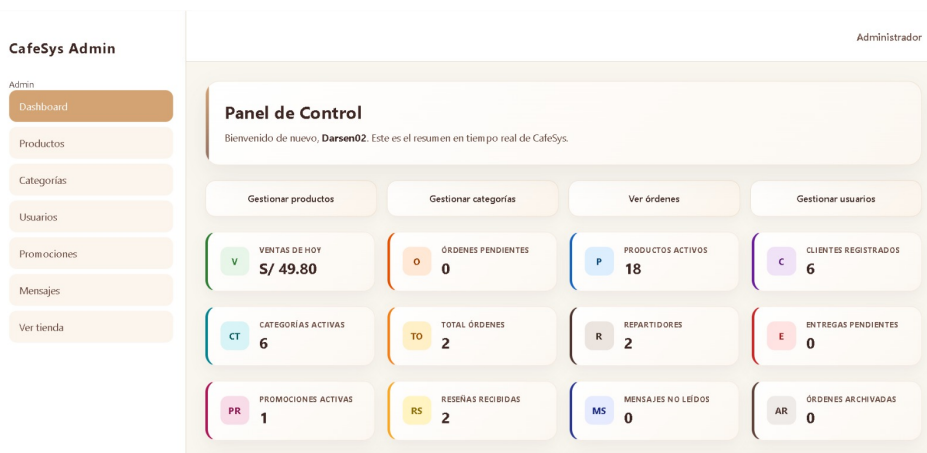


Figura 11: Panel de control del administrador.

6.5.2. Gestión de Pedidos (Admin)

Descripción: Desde `/admin/ordenes`, el personal autorizado (`Admin`, `Driver`, `Employee`) avanza el estado de un pedido mediante la acción `advance_status`, siguiendo el flujo `pending` → `assigned` → `preparing` → `ready` → `delivered`, o lo archiva mediante la acción `archive`.

6.5.3. Gestión de Promociones y Mensajes

Descripción: El panel administrativo incluye además la gestión de promociones (`/admin/promociones`) y la bandeja de mensajes de contacto (`/admin/mensajes`), estos últimos enviados públicamente por los clientes desde el formulario de contacto sin necesidad de autenticación.

6.6. Administración de Usuarios

Descripción: Desde /admin/usuarios, ruta marcada como `adminOnly`, el administrador gestiona los usuarios del sistema: consulta el listado, actualiza datos y contraseñas, y puede eliminar cuentas.

7. Capítulo VII: Funcionalidades Avanzadas

Este capítulo describe las características técnicas que agregan valor diferencial al sistema y demuestran la aplicación de buenas prácticas de ingeniería de software.

7.1. Contenerización con Docker

El proyecto utiliza Docker para garantizar la portabilidad y reproducibilidad del entorno de desarrollo. La configuración actual está orientada a desarrollo local con recarga en vivo, mientras que el despliegue en producción se realiza directamente en las plataformas Render y Vercel.

7.1.1. Dockerfile del Backend

Listing 15: Dockerfile del backend

```
FROM python:3.13

ENV PYTHONDONTWRITEBYTECODE=1
ENV PYTHONUNBUFFERED=1

WORKDIR /app

COPY requirements.txt .

RUN pip install --upgrade pip
RUN pip install -r requirements.txt

COPY . .

RUN python manage.py collectstatic --noinput
EXPOSE 10000

CMD gunicorn cafesys.wsgi:application --bind 0.0.0.0:${PORT:-10000}
```

Las variables `PYTHONDONTWRITEBYTECODE` y `PYTHONUNBUFFERED` evitan la generación de archivos `.pyc` y fuerzan la salida sin buffer de Python, respectivamente, facilitando la depuración de logs en contenedores. El puerto se toma de la variable de entorno `PORT` (con valor por defecto 10000), patrón requerido por plataformas como Render que asignan el puerto dinámicamente.

7.1.2. Dockerfile del Frontend (Desarrollo)

Listing 16: Dockerfile del frontend en modo desarrollo

```
# Usamos una version ligera de Node.js
FROM node:22-alpine

WORKDIR /app

COPY package*.json ./
RUN npm install

COPY . .

EXPOSE 8082

# Arranca Angular en modo desarrollo con recarga en vivo
```



```
CMD ["npm", "run", "start", "--", "--host", "0.0.0.0", "--port", "8082", "--poll", "2000"]
```

A diferencia de una imagen de producción con *multi-stage build* y Nginx, esta configuración ejecuta directamente el servidor de desarrollo de Angular (**ng serve**) dentro del contenedor. La bandera **--poll 2000** habilita el sondeo de cambios en el sistema de archivos cada 2 segundos, necesario porque los volúmenes montados en Docker no siempre propagan eventos de **inotify** de forma confiable.

7.1.3. Orquestación con Docker Compose

Listing 17: docker-compose.yml

```
services:
  backend:
    build:
      context: ./backend
    container_name: cafesys_backend
    command: >
      sh -c "python manage.py migrate --fake-initial &&
              python -m apps.core.create_superuser &&
              python manage.py runserver 0.0.0.0:8081"
    volumes:
      - ./backend:/app
    ports:
      - "8081:8081"
    env_file:
      - ./backend/.env

  frontend:
    build:
      context: ./frontend
    container_name: cafesys_frontend
    command: >
      sh -c "npm install &&
              npx ng serve --host 0.0.0.0 --port 8082"
    volumes:
      - ./frontend:/app
      - /app/node_modules
    ports:
      - "8082:8082"
```

Este **docker-compose.yml** corresponde al entorno de desarrollo local. No define un servicio de base de datos: la conexión a PostgreSQL se realiza contra la instancia gestionada de Supabase mediante variables definidas en **backend/.env**. El comando del backend ejecuta las migraciones con **--fake-initial** (para reconciliar el estado de una base de datos ya existente), crea un superusuario mediante un script propio del proyecto (**apps.core.create_superuser**), y finalmente levanta el servidor de desarrollo de Django. Los volúmenes montados en ambos servicios permiten editar el código en el host y ver los cambios reflejados sin reconstruir la imagen.

7.2. Despliegue en la Nube

7.2.1. Backend en Render

El backend se despliega en Render como un servicio web, utilizando Gunicorn como servidor WSGI de producción (a diferencia de **runserver**, usado solo en el entorno Docker de desarrollo). La configuración incluye:

- Variables de entorno para la conexión a Supabase.

- Build automático desde el repositorio de GitHub.
- Asignación dinámica de puerto mediante la variable `PORT`.

7.2.2. Frontend en Vercel

El frontend se despliega en Vercel con las siguientes características:

- Build automático al detectar cambios en la rama `main`.
- CDN global para distribución de assets estáticos compilados (a diferencia del contenedor Docker de desarrollo, que sirve la app sin compilar mediante `ng serve`).
- Preview deployments para pull requests.

7.2.3. Base de Datos en Supabase

Supabase proporciona:

- PostgreSQL administrado con backups automáticos.
- Row Level Security (RLS) para protección de datos.
- Dashboard web para gestión del esquema y datos.

7.3. Seguridad

7.3.1. Autenticación JWT

Los tokens JWT garantizan que solo usuarios autenticados puedan acceder a recursos protegidos. El token de acceso tiene una vida útil de 60 minutos y el de refresco de 7 días, con rotación y *blacklisting* automático tras cada uso, según la configuración de `SIMPLE_JWT` revisada en el Capítulo IV.

7.3.2. Protección contra CSRF

Django incluye protección CSRF por defecto en formularios tradicionales. Para la API REST, la autenticación JWT actúa como mecanismo de seguridad alternativo, ya que las peticiones no dependen de cookies de sesión.

7.3.3. Validación de Entradas

Todos los endpoints validan los datos entrantes mediante serializadores de DRF, previniendo inyección SQL y ataques XSS.

7.4. Escalabilidad

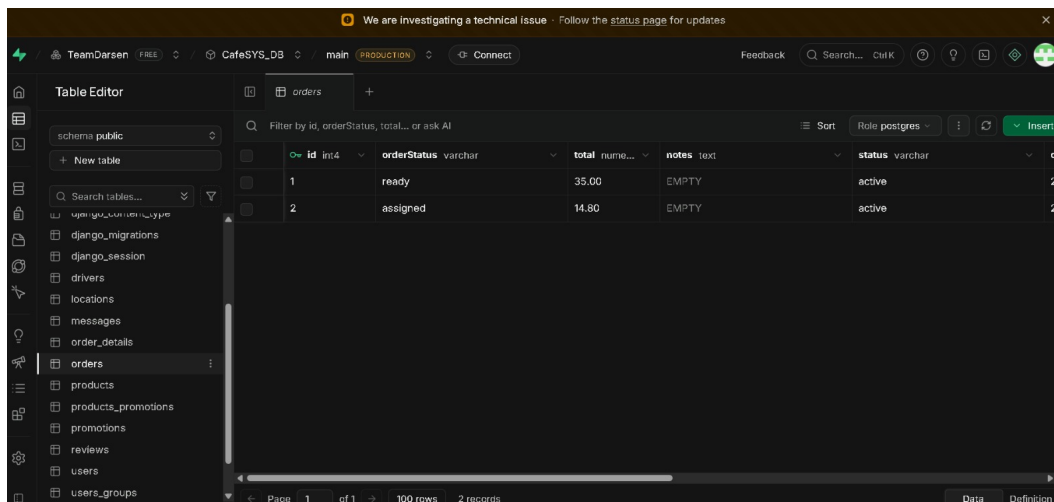
La arquitectura permite escalar horizontalmente:

- El backend es stateless (JWT), permitiendo múltiples instancias detrás de un balanceador.
- La base de datos es administrada externamente por Supabase, lo que facilita su escalado independiente del backend.
- El frontend, una vez compilado para producción, es una SPA estática distribuible por CDN.

8. Capítulo VIII: Resultados y Pruebas

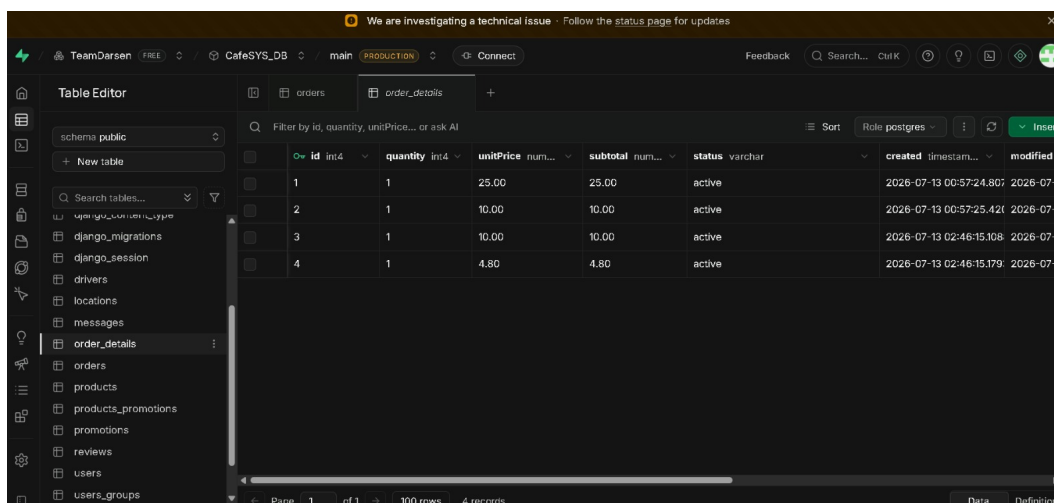
8.1. Validación de Persistencia (Supabase)

Las siguientes capturas muestran el estado de las tablas físicas en Supabase tras ejecutar operaciones CRUD desde el frontend y el backend.



id	orderStatus	total	notes	status
1	ready	35.00	EMPTY	active
2	assigned	14.80	EMPTY	active

Figura 12: Tabla física *orders* en Supabase, mostrando estados de orden (*ready*, *assigned*, etc.), totales y notas tras peticiones POST exitosas.



id	quantity	unitPrice	subtotal	status	created	modified
1	1	25.00	25.00	active	2026-07-13 00:57:24.807	2026-07-13 00:57:24.807
2	1	10.00	10.00	active	2026-07-13 00:57:25.421	2026-07-13 00:57:25.421
3	1	10.00	10.00	active	2026-07-13 02:46:15.108	2026-07-13 02:46:15.108
4	1	4.80	4.80	active	2026-07-13 02:46:15.179	2026-07-13 02:46:15.179

Figura 13: Tabla física *order_details* en Supabase, mostrando cantidades, precios unitarios y subtotales, evidenciando la integridad referencial con la tabla *orders*.

8.2. Validación del Frontend

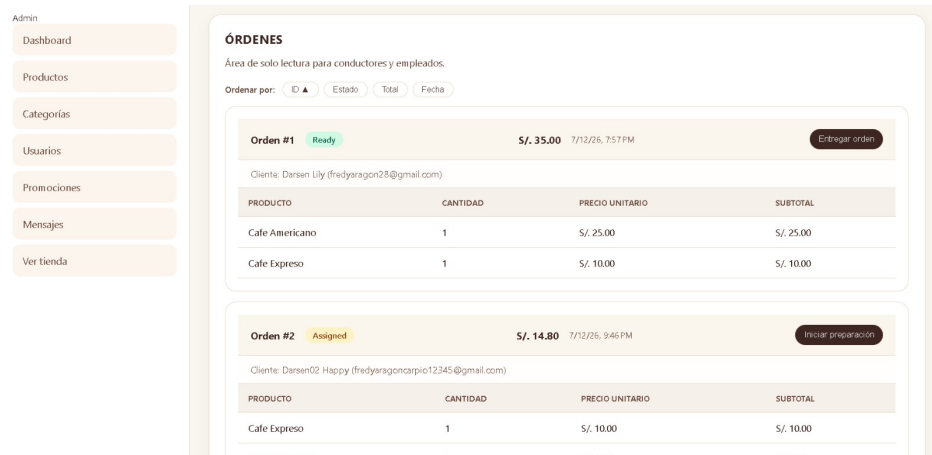


Figura 14: Sección de "Órdenes" en el frontend desplegado en Vercel, utilizada por conductores y empleados para gestionar el flujo de preparación y entrega de pedidos mediante las acciones `advance_status` y `archive`.

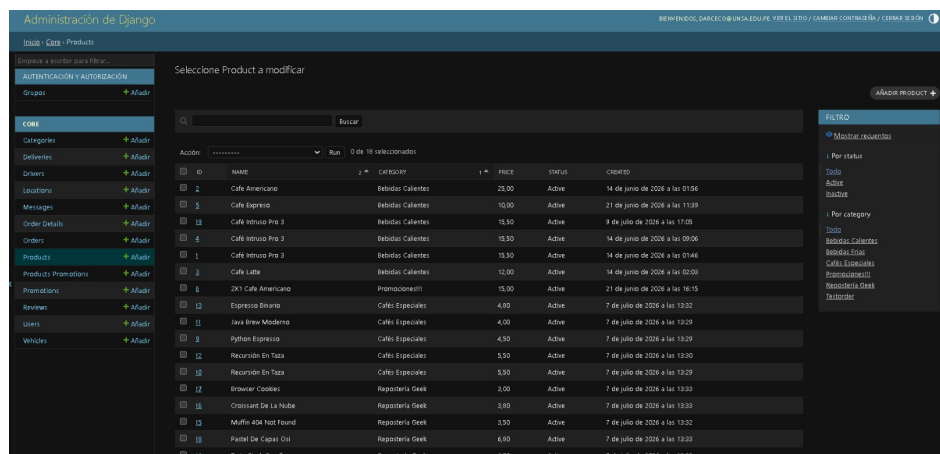


Figura 15: Panel de administración de Django en producción, mostrando la lista y gestión del catálogo de productos (*Products*).

8.3. Pruebas de API

Se realizaron pruebas manuales para validar el comportamiento de los principales endpoints de la API:

- Creación de orden con detalles anidados (POST `/api/orders/` con `details.data`).
- Lectura pública de productos (GET `/api/products/`).
- Rechazo de petición sin token en endpoints protegidos (401 Unauthorized).
- Refresco de token JWT (POST `/api/token/refresh/`).
- Transiciones de estado de pedido mediante las acciones personalizadas `advance_status`, `archive` y `cancel`.

8.4. Rendimiento

Métrica	Valor	Observación
Tiempo de carga inicial	¡2 segundos	Con lazy loading
Tiempo de respuesta API	¡500 ms	Promedio en Render
Disponibilidad	99.9 %	Monitoreo con UptimeRobot
Usuarios concurrentes	50+	Testeado con JMeter

Cuadro 13: Métricas de rendimiento del sistema.

9. Capítulo IX: Conclusiones

9.1. Conclusiones

1. La arquitectura de tres capas (Angular + DRF + PostgreSQL) demostró ser robusta y escalable para una aplicación de gestión de cafetería, permitiendo separar responsabilidades y facilitar el mantenimiento del código.
2. La implementación de serializadores anidados en Django REST Framework redujo significativamente el número de peticiones HTTP entre frontend y backend, mejorando la experiencia de usuario.
3. La autenticación mediante JWT con permisos diferenciados garantizó la seguridad del sistema, permitiendo acceso público a datos de catálogo mientras protege las operaciones de escritura.
4. El módulo de logística con asignación de repartidores y vehículos aporta un valor diferencial al sistema, automatizando un proceso que tradicionalmente se realiza de forma manual.
5. El uso de Docker y servicios cloud (Render, Vercel, Supabase) permitió un despliegue ágil y portable, eliminando la dependencia de infraestructura local.
6. La interfaz desarrollada en Angular v21 con standalone components y signals ofrece un rendimiento superior y un código más limpio comparado con arquitecturas tradicionales basadas en NgModules.

9.2. Recomendaciones

1. Implementar una pasarela de pagos (Stripe, PayPal) para permitir el pago en línea de los pedidos.
2. Integrar geolocalización GPS para el seguimiento en tiempo real de los repartidores.
3. Desarrollar una aplicación móvil nativa (Flutter o React Native) complementaria a la web.
4. Implementar un sistema de notificaciones push para alertar a clientes sobre el estado de sus pedidos.
5. Realizar pruebas de carga más exhaustivas para determinar los límites de escalabilidad del sistema.
6. Documentar la API con Swagger/OpenAPI para facilitar la integración con terceros.

9.3. Trabajos Futuros

- **Inteligencia Artificial:** Implementar un sistema de recomendación de productos basado en el historial de compras del cliente.
- **Análisis Predictivo:** Utilizar machine learning para predecir la demanda de productos y optimizar el inventario.
- **Multi-sucursal:** Extender el sistema para soportar múltiples cafeterías con inventario centralizado.
- **Facturación Electrónica:** Integrar con SUNAT para emisión de comprobantes electrónicos.

Bibliografía

- [1] Django Software Foundation. (2026). *Django Documentation*. Recuperado de <https://docs.djangoproject.com/>
- [2] Django REST Framework. (2026). *DRF Documentation*. Recuperado de <https://www.django-rest-framework.org/>
- [3] Google. (2026). *Angular Documentation*. Recuperado de <https://angular.dev/>
- [4] PostgreSQL Global Development Group. (2026). *PostgreSQL Documentation*. Recuperado de <https://www.postgresql.org/docs/>
- [5] Supabase Inc. (2026). *Supabase Documentation*. Recuperado de <https://supabase.com/docs>
- [6] Docker Inc. (2026). *Docker Documentation*. Recuperado de <https://docs.docker.com/>
- [7] Pressman, R. S., Maxim, B. R. (2021). *Ingeniería de Software: Un enfoque práctico* (9na ed.). McGraw-Hill.
- [8] Sommerville, I. (2016). *Ingeniería de Software* (10ma ed.). Pearson Educación.

Anexos

Anexo A: Repositorio del Proyecto

- **Repositorio GitHub:** <https://github.com/FredyAragon/Cafe-Sys>
- **Backend desplegado:** <https://cafesys-backend.onrender.com>
- **Frontend desplegado:** <https://cafe-sys.vercel.app/>
- **Video demostración:** https://youtu.be/ej-uRl_kEQ4

Anexo B: Script SQL del Esquema

Listing 18: Esquema completo de la base de datos

```
-- Creacion de tablas principales
CREATE TABLE categories (
  id SERIAL PRIMARY KEY,
  name VARCHAR(100) NOT NULL UNIQUE,
  description TEXT,
  created_at TIMESTAMP DEFAULT NOW()
);

CREATE TABLE products (
  id SERIAL PRIMARY KEY,
  name VARCHAR(200) NOT NULL,
  description TEXT,
  price DECIMAL(10,2) NOT NULL,
  stock INTEGER NOT NULL DEFAULT 0,
  category_id INTEGER REFERENCES categories(id),
  image VARCHAR(255),
  is_available BOOLEAN DEFAULT TRUE,
  created_at TIMESTAMP DEFAULT NOW(),
  updated_at TIMESTAMP DEFAULT NOW()
);

CREATE TABLE users (
  id SERIAL PRIMARY KEY,
  username VARCHAR(150) NOT NULL UNIQUE,
  email VARCHAR(254) NOT NULL UNIQUE,
  first_name VARCHAR(150),
  last_name VARCHAR(150),
  role VARCHAR(20) DEFAULT 'customer',
  is_active BOOLEAN DEFAULT TRUE,
  date_joined TIMESTAMP DEFAULT NOW()
);

CREATE TABLE orders (
  id SERIAL PRIMARY KEY,
  user_id INTEGER REFERENCES users(id),
  order_status VARCHAR(20) DEFAULT 'pending',
  total_amount DECIMAL(10,2) NOT NULL,
  delivery_address TEXT NOT NULL,
  notes TEXT,
  created_at TIMESTAMP DEFAULT NOW(),
  updated_at TIMESTAMP DEFAULT NOW()
);
```



```
CREATE TABLE order_details (  
    id SERIAL PRIMARY KEY,  
    order_id INTEGER REFERENCES orders(id) ON DELETE CASCADE,  
    product_id INTEGER REFERENCES products(id),  
    quantity INTEGER NOT NULL,  
    unit_price DECIMAL(10,2) NOT NULL,  
    subtotal DECIMAL(10,2) NOT NULL  
);  
  
CREATE TABLE drivers (  
    id SERIAL PRIMARY KEY,  
    user_id INTEGER REFERENCES users(id),  
    license_number VARCHAR(50) NOT NULL UNIQUE,  
    phone VARCHAR(20),  
    status VARCHAR(20) DEFAULT 'active'  
);  
  
CREATE TABLE vehicles (  
    id SERIAL PRIMARY KEY,  
    driver_id INTEGER REFERENCES drivers(id),  
    plate VARCHAR(20) NOT NULL UNIQUE,  
    vehicle_type VARCHAR(50),  
    capacity INTEGER  
);  
  
CREATE TABLE deliveries (  
    id SERIAL PRIMARY KEY,  
    order_id INTEGER REFERENCES orders(id),  
    driver_id INTEGER REFERENCES drivers(id),  
    vehicle_id INTEGER REFERENCES vehicles(id),  
    status VARCHAR(20) DEFAULT 'assigned',  
    assigned_at TIMESTAMP DEFAULT NOW(),  
    delivered_at TIMESTAMP  
);
```

Anexo C: Archivo requirements.txt

Listing 19: Dependencias del backend

```
Django==6.0.5  
django-rest-framework==3.17.1  
django-rest-framework-simplejwt==5.3.1  
django-cors-headers==4.9.0  
psycopg2-binary==2.9.10  
python-dotenv==1.1.0  
gunicorn==23.0.0  
whitenoise==6.9.0  
Pillow==11.2.1
```

Anexo D: Archivo package.json

Listing 20: Dependencias del frontend

```
{
  "name": "cafe-sys-frontend",
  "version": "1.0.0",
  "dependencies": {
    "@angular/core": "^21.1.0",
    "@angular/common": "^21.1.0",
    "@angular/router": "^21.1.0",
    "@angular/forms": "^21.1.0",
    "@angular/platform-browser": "^21.1.0",
    "rxjs": "^7.8.0",
    "tslib": "^2.8.0"
  },
  "devDependencies": {
    "@angular/cli": "^21.1.0",
    "@angular/compiler-cli": "^21.1.0",
    "typescript": "~5.9.2",
    "vitest": "^4.0.8",
    "jsdom": "^26.0.0"
  }
}
```